

Scaling Scenario: 10,000 Concurrent Users

Scenario

Support 10,000 concurrent users submitting lists of up to 1,000 numbers each, plus two new capabilities: save/retrieve named lists, and an audit log of all submissions by user.

Key Insight: Memoization

The conversion function is pure — same input always yields the same output. A `number_words` table stores every number we've ever converted exactly once. All saved lists reference these rows by FK rather than duplicating text. Data is immutable, so caches never need invalidation and the system gets faster with use.

Frontend/State Management Updates

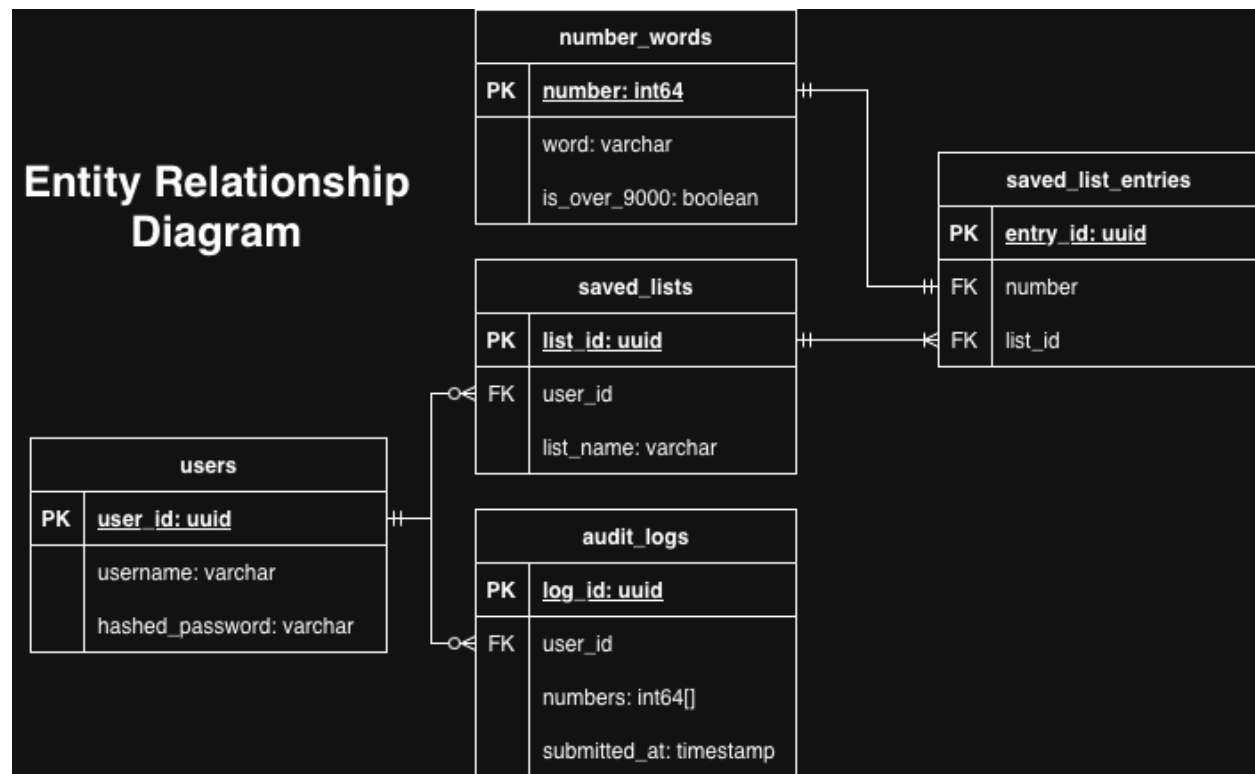
- The current frontend is a single page with local component state (`useState` in `Home.tsx`). To support auth, saved lists, and audit history, the following changes are needed:
 - **Auth state:** Introduce a global auth context (react context) to hold the JWT, user info, and login/logout actions. All API calls attach the token via a fetch wrapper.
 - **Routing:** Add react-router for navigation between pages — Home page, and audit history for now. The current single-page layout won't scale to multiple features.
 - **Saved lists UI:**
 - a button on/near the `ResultsList` component that says “Save List”, that fires the new `PUT /api/lists/{name}` request.
 - A new `SavedLists.tsx` component that shows the name of the lists that user has saved. Selecting a saved list name reloads the `ResultsList` with the saved list and clears the input.
 - **Audit history UI:** Paginated table view with date-range filters. State managed locally would include page number, records on that page, and filter values.
 - **Loading/error states per feature:** Each new feature (lists, audit) needs its own loading and error state, rather than sharing the single `isLoading/requestError` pair currently used for conversions. As we add more endpoints, we should switch over to React Query to handle these fields for each API call our frontend makes with built in framework support and response caching.
 - **Token refresh handling:** A silent refresh mechanism (refresh token stored in an `httpOnly` cookie) to avoid forcing re-login during active sessions.

API Design

Operation	Method & Path	Notes
Convert numbers	POST /api/numberwords	Check cache → convert misses → store new entries → return. Audit written async.
Save a list	PUT /api/lists/{name}	Upsert list + entries referencing number_words
Retrieve a list	GET /api/lists/{name}	JOIN entries → number_words. No recomputation.
List saved lists	GET /api/lists	Metadata only (name, count, timestamps)
Audit history	GET /api/audit?from=&to=&page=	Paginated on (user_id, submitted_at DESC)
Sign up	POST /api/auth/register	Accepts username + password, returns access token + refresh token
Log in	POST /api/auth/login	Validates credentials, returns access token + refresh token
Refresh token	POST /api/auth/refresh	Accepts refresh token, returns new access token

**All list and audit endpoints scope queries by user_id extracted from the JWT sub claim — no user ID is passed in the request body or path.

Database Design



Scaling Strategy / Tradeoffs

- **Conversion stays fast:** The response never waits on a DB write. Audit and new memoization entries are written asynchronously. Cached values are looked up instantly so the performance improves over time.
- **Caching:** In-memory cache (ConcurrentDictionary) → number_words table → compute on miss. Immutable data = no invalidation. If memory becomes a concern, could use TTL or LRU eviction strategies.
- **Connection pooling:** At 10K concurrent users, DB connections are the bottleneck. Async data access + connection pooler between app and DB. Worst case for huge scale writes like this if we didnt have the memoized DB solution we could use SQS or some message queue service to queue the writes, like if audit logs was getting 50k requests/second and we started seeing increased latency to where some operations failed or timed out then a queue would be the best solution. Limitation there would be eventual consistency.
- **Audit log growth:** Partition by submitted_at (monthly). Store only raw input numbers (not converted output) to keep rows lean.
- **Auth:** JWT-based, stateless. Short-lived access tokens validated by signature only — no DB lookup per request, keeping the connection pool free.